MPI Overview

The Message Passing Interface, MPI in short, is the most widely used standard for distributed and parallel computing on HPC clusters. MPI standardizes collaboration through **communicator** objects between **processes** that run in parallel on one or on separate compute nodes. We'll focus on MPI 5.0 and the world model (there's also the session model) that predefines the `MPI_COMM_WORLD` communicator object. This object is comprised of all processes. Processes are numbered sequentially by **rank**, starting with 0.

Several implementations of MPI exist. OpenMPI, MPICH, and MVAPICH are well known, vendor based implementations usually are specializations of these - Intel MPI, Cray MPI, …

Communicator objects come in two flavours. The first, like `MPI_COMM_WORLD`, are **intra-communicators** used to communicate between the processes in a **group**. The second, **inter-communicators**, are for communicating between distinct groups of processes. In the following, we'll focus on intra-communicators.

Intra-communicator objects are associated with

- **context** for communication (e.g. messages within a context are ordered according to some rules)
  - messages sent from one rank to another one are received in the order they were sent
- **group** of processes, ordered by sequentially increasing integer rank (starting with 0)
  - groups with the same processes, but different ordering, are not exactly equal
- **topology**, virtual neighborhood information
  - like cartesian, or graph topologies
  - topologies map ranks to coordinates or graph vertices
- **attributes**, (tag, value) pairs for additional info
  - e.g. `MPI_COMM_WORLD` has the `MPI_TAG_UB` attribute (upper bound for tags)
- **error handler**
  - by default, the predefined `MPI_ERRORS_ARE_FATAL` handler is set
  - the predefined `MPI_ERRORS_RETURN` handler can be set to handle errors explicitly in the code

Communicators can be duplicated or split into subcommunicators with distinct groups in various ways. It's also possible to create new communicators specifying a group of processes. Different communicators can be used independently.

The predefined intra-communicator `MPI_COMM_SELF` only has the local rank in the group.

Source and destination

If source or destination are needed, they are specified by the process rank. `MPI_ANY_SOURCE` accepts incoming data from any rank. `MPI_PROC_NULL` is a dummy rank, that may be valid as source or destination in order to simplify the code.

Point to point communication

Send and **receive** commands will see data in sending order on the receiver within a communication context. The datatypes must match on both sides. The receiver can specify a higher element count than the sender, but not vice versa.

Send operations specify a **destination** (or `MPI_PROC_NULL`) rank and an integer **tag**. Receive operations specify the **source** (or `MPI_ANY_SOURCE` / `MPI_PROC_NULL`) rank and an integer **tag** (or `MPI_ANY_TAG`). The source and destination ranks and the tags must match.

Receive operations have a **status** object output argument, it is filled with info on the number of transferred elements, the source rank, the tag, or an error on failure. `MPI_STATUS_IGNORE` may be given as argument if the status is unimportant.

Send and receive exist in many different versions, discernible by prefix and _suffix.

Mode	Blocking	Non-blocking	Persistent
Normal		I	_init
Buffered	B	IB	B _init
Synchronous	S	IS	S _init
Ready	R	IR	R _init
Partitioned	-	-	P _init

Versions

- **Blocking**: Sender waits until the send buffer can be used again. Receiver waits until the data is received.
- **Non-blocking**: Start the operation in the background. Return a **request** object that MUST be queried for completion or waited for in order to finish the operation. Great for hiding communication.
- **Persistent**: Prepare the communication operation and return a request object. The request can then be started later and repeatedly via this object. After starting the operation, the request is used as in non-blocking communication. Might save time if the same operation is done many times.

Modes

- **Normal**: The system decides how this is done
- **Buffered**: Data is buffered. System sends data in the background.
- **Synchronous**: Wait until receiver starts receiving and send buffer is no longer used.
- **Ready**: Receiver MUST be ready before send. No buffering is allowed.
- **Partitioned**: Send/receive data in chunks. Chunk size must not match.

Collective communication

Collective operations must be done by all ranks in the group associated with an intra-communicator. They exist in normal, non-blocking, and persistent versions.

- **Barrier**: synchronization
- **Broadcast**: one to all
- **Gather**: all to one
- **Scatter**: distribute content from one to all
- **Allgather**: like gather, but all receive the result
- **Alltoall**: like scatter, but receive from all
- **Reduction**: reduction from all to one
  - maximum, minimum
  - sum, product
  - logical and, bit-wise and
  - logical or, bit-wise or
  - logical xor, bit-wise xor
  - max, min value and location
- **Allreduce**: like reduce, but all receive the result
- **Reduce-Scatter**: like reduce, but result is distributed to all

- **Scan**: prefix reduction, rank i recives reduction result from rank $0..i$

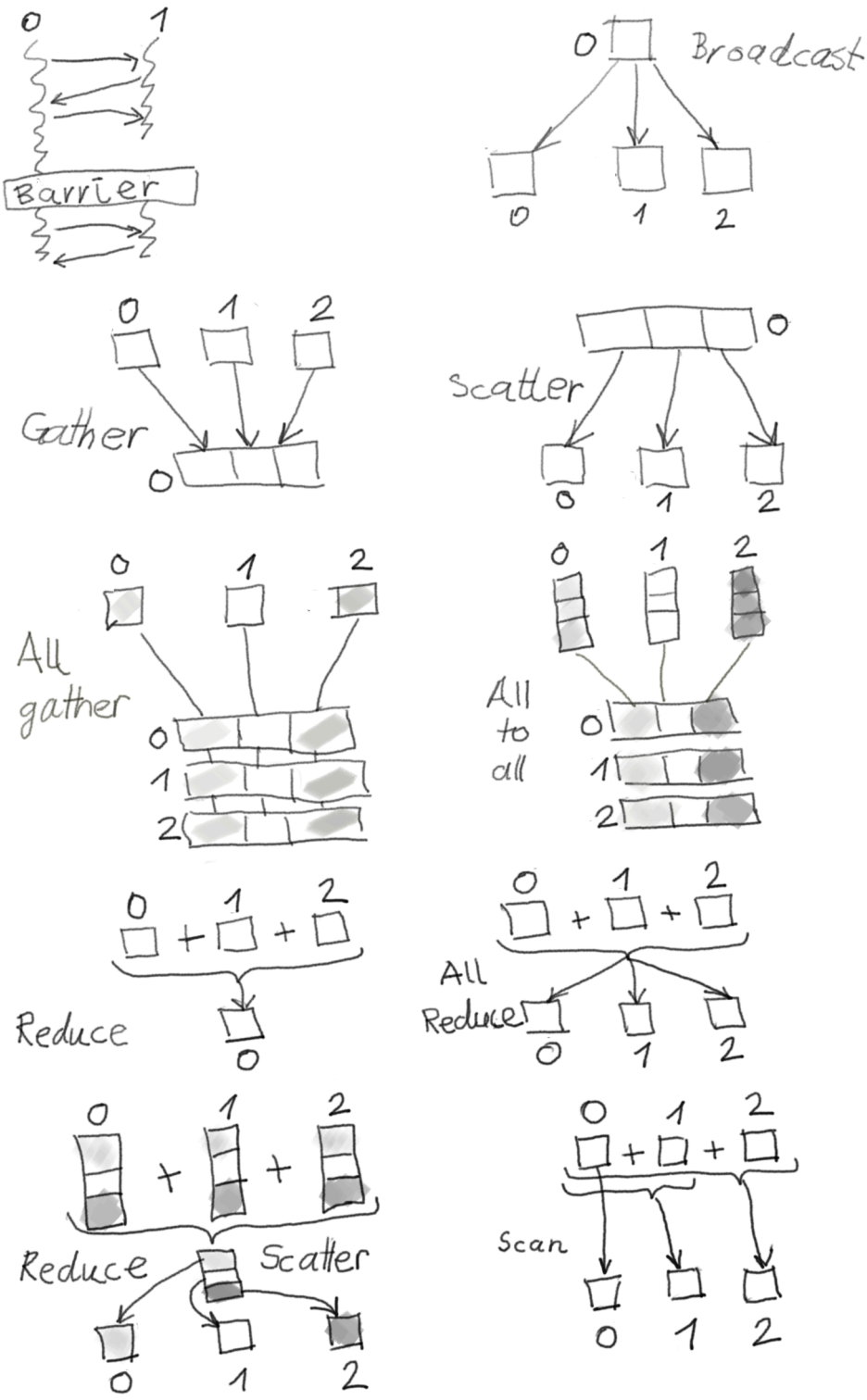

Figure 1: Collectives

Virtual topologies

Create neighbourhood links and communicate with neighbours.

- Create cartesian topology
- Create graph topology
- Neighbourhood gather
- Neighbourhood alltoall

One sided communication

Remote memory window access and operations, separates communication from synchronization.

- Read
- Write
- Accumulate
- Read and update
- Compare and swap

Datatypes

You can create your own structured and array data types.

I/O

MPI supports parallel I/O. Libraries like HDF5 support MPI I/O.

A file is abstracted as a view on a sequence of elementary data type values. Both collective and individual access functions are standardized, as well as blocking and non-blocking file access.

Depending on the underlying file system and implementation, MPI I/O supports

- collective buffering: I/O is performed by a subset of compute nodes that collect smaller chunks into bigger ones
- striped access: file is distributed to stripes on ditinct I/O devices to increase throughput
- data access through chunks: like subarrays

Dynamic processes

Create new processes and communicators for interacting with them.

References

- MPI Forum (https://www.mpi-forum.org)
- OpenMPI (https://www.open-mpi.org)
- MPICH (https://www.mpich.org)